n8n Quick Reference Guide

Core Concepts

What is n8n?

n8n (pronounced "n-eight-n") is a fair-code licensed workflow automation tool that connects apps and services with minimal coding. It combines visual workflow building with powerful code capabilities.

Key Components

1. Workflows

A collection of nodes connected together to automate a process.

2. Nodes

Building blocks of workflows. Three main types:

- **Trigger Nodes** ⚡: Start workflows (webhook, schedule, email trigger)
- **Action Nodes**: Perform tasks (API calls, data processing, file operations)
- **Core Nodes**: Provide logic and control (IF, Switch, Merge, Code)

3. Connections

Links between nodes that define data flow and execution order.

4. Executions

Instances of workflow runs (manual, scheduled, or triggered).

5. Credentials

Secure storage for API keys, passwords, and authentication tokens.

Essential Node Types

Trigger Nodes

Node	Purpose	Use Case
Webhook	Receive HTTP requests	API endpoints, form submissions
Schedule	Run on schedule	Daily reports, periodic cleanup
Manual Trigger	Start manually	Testing, on-demand tasks
Error Trigger	Catch workflow errors	Error handling workflows
Email Trigger	Trigger on email	Email processing automation

Core Logic Nodes

Node	Purpose	Use Case
IF	Conditional branching	Route data based on conditions
Switch	Multiple conditions	Complex routing logic
Code	JavaScript/Python	Custom data transformation
Merge	Combine data streams	Join data from multiple sources
Split In Batches	Process large datasets	Handle pagination, batch processing

Data Transformation

Node	Purpose	Use Case
Set	Create/modify fields	Prepare data for next node
Edit Fields	Rename, remove fields	Clean up data structure
Aggregate	Summarize data	Count, sum, group data
Filter	Remove unwanted items	Data filtering

Integration Nodes

Category	Examples
Databases	PostgreSQL, MySQL, MongoDB, Supabase
Communication	Gmail, Slack, Telegram, Discord
Storage	Google Drive, Dropbox, AWS S3
CRM	HubSpot, Salesforce, Pipedrive
AI	OpenAI, AI Agent, Google Gemini

Data Structure & Expressions

n8n Data Format

Data flows between nodes as **items** (array of JSON objects):

JSON

```
[
  {
    "json": {
      "name": "John",
      "email": "john@example.com"
    }
  },
  {
    "json": {
```

```
      "name": "Jane",
      "email": "jane@example.com"
    }
  }
]
```

Expression Syntax

Access data using double curly braces: `{{ expression }}`

Common Expressions:

JavaScript

```
// Current item data
{{ $json.fieldName }}

// All items from current node
{{ $input.all() }}

// Data from specific node
{{ $node["NodeName"].json.fieldName }}

// First item from previous node
{{ $input.first().json.fieldName }}

// Item index
{{ $itemIndex }}

// Workflow metadata
{{ $workflow.name }}
{{ $execution.id }}

// Date/time
{{ $now }}
{{ $today }}
```

Useful Functions:

JavaScript

```
// String operations
{{ $json.text.toLowerCase() }}
{{ $json.text.trim() }}
{{ $json.text.split(',') }}

// Array operations
```

```
{{ $json.items.length }}
{{ $json.items.map(item => item.name) }}
{{ $json.items.filter(item => item.active) }}

// Math
{{ Math.round($json.value) }}
{{ Math.max(...$json.numbers) }}

// Date formatting
{{ $now.toFormat('yyyy-MM-dd') }}
{{ $now.plus({ days: 7 }) }}
```

Error Handling

Best Practices

1. **Create Error Workflows**: Use Error Trigger node
2. **Set Error Workflow**: In workflow settings
3. **Use Continue On Fail**: For non-critical nodes
4. **Implement Retry Logic**: For API calls
5. **Add Stop And Error**: For controlled failures

Error Workflow Pattern

Plain Text

```
Main Workflow (Settings → Error Workflow = "Error Handler")
  ↓ (if error occurs)
Error Handler Workflow
  → Error Trigger
  → Log Error (to database/file)
  → Send Notification (Slack/Email)
  → Optional: Retry Logic
```

Retry Configuration

In node settings:

- **Retry On Fail**: Enable
- **Max Tries**: 3-5
- **Wait Between Tries**: 1000ms (exponential backoff)

Workflow Optimization

Performance Tips

1. **Batch API Calls**: Use Split In Batches for large datasets
2. **Minimize Nodes**: Use Code node for complex transformations
3. **Parallel Execution**: Split workflows into parallel branches
4. **Cache Results**: Store frequently accessed data
5. **Limit Data**: Only fetch/process what you need

Execution Modes

- **Manual**: Test during development
- **Production**: Activated workflows run automatically
- **Partial**: Test specific parts of workflow

Common Workflow Patterns

1. Webhook API Endpoint

Plain Text

```
Webhook Trigger
  → Validate Input (IF node)
  → Process Data (Code/HTTP Request)
  → Return Response (Respond to Webhook)
  → Log Result (Database/File)
```

2. Scheduled Data Sync

Plain Text

```
Schedule Trigger (daily at 9 AM)
  → Fetch Data (HTTP Request/Database)
  → Transform Data (Code/Set)
  → Update Destination (API/Database)
  → Send Summary (Email/Slack)
```

3. AI-Powered Automation

```
Trigger (Webhook/Chat)
  → Prepare Context (Set/Code)
  → AI Agent (with tools)
  → Process Response (Code)
  → Take Action (API calls)
  → Return Result
```

4. Error Recovery

```
Main Workflow
  → Risky Operation (Continue On Fail: OFF)
  → Success Path

Error Workflow (triggered on failure)
  → Error Trigger
  → Analyze Error (Code)
  → Retry or Alert
```

Debugging Tips

1. Use Execution View

- Click on nodes to see input/output data
- Check for empty results or unexpected data types
- Verify expressions are evaluating correctly

2. Test Incrementally

- Build workflows step-by-step
- Test each node before adding the next
- Use Manual Trigger for testing

3. Common Issues

Problem	Likely Cause	Solution
No data passing	Empty result from previous node	Check previous node output
Expression error	Wrong syntax or undefined field	Verify field names, use $json
Workflow fails	API error or timeout	Add error handling, retry logic
Slow execution	Too many sequential calls	Batch operations, parallel execution

4. Enable Debug Mode

- Check execution logs
- Use console.log() in Code nodes
- Enable "Save Execution Progress" in settings

Security Best Practices

Credentials Management

- ✅ Store API keys in n8n credentials
- ✅ Use environment variables for config
- ❌ Never hardcode secrets in workflows
- ❌ Don't expose credentials in logs

Webhook Security

- Use authentication (header auth, basic auth)
- Validate incoming data
- Rate limit requests
- Use HTTPS only

Data Privacy

- Minimize data retention

- Encrypt sensitive data
- Follow GDPR/compliance requirements
- Regular credential rotation

Useful Resources

Official Documentation

- **Main Docs**: https://docs.n8n.io/
- **Node Reference**: https://docs.n8n.io/integrations/builtin/
- **Workflow Templates**: https://n8n.io/workflows/
- **Community Forum**: https://community.n8n.io/

Learning Resources

- **Quickstart Guide**: https://docs.n8n.io/getting-started/quickstart/
- **Video Courses**: https://docs.n8n.io/courses/
- **AI Tutorial**: https://docs.n8n.io/advanced-ai/intro-tutorial/

Community

- **Forum**: Ask questions, share workflows
- **GitHub**: Report bugs, request features
- **Discord**: Real-time community support

Version Differences

n8n Cloud

- Managed hosting
- Automatic updates
- Built-in scaling
- No infrastructure management

Self-Hosted

- Full control

- Custom nodes
- On-premise deployment
- Manual updates

Feature Availability

Some features require specific versions or plans:

- **AI Nodes**: v1.19.4+
- **Projects**: Enterprise feature
- **LDAP/SAML**: Enterprise feature
- **Log Streaming**: Enterprise feature

Keyboard Shortcuts

Action	Shortcut
Search nodes	`/`
Execute workflow	`Ctrl/Cmd + Enter`
Save workflow	`Ctrl/Cmd + S`
Delete node	`Delete`
Duplicate node	`Ctrl/Cmd + D`
Select all	`Ctrl/Cmd + A`
Undo	`Ctrl/Cmd + Z`
Redo	`Ctrl/Cmd + Shift + Z`

Quick Troubleshooting Checklist

When a workflow isn't working:

- [ ] Check all nodes are connected properly
- [ ] Verify credentials are set and valid

- [ ] Review execution data for each node
- [ ] Check expressions for syntax errors
- [ ] Ensure previous nodes return expected data
- [ ] Look for error messages in execution logs
- [ ] Test with manual trigger first
- [ ] Verify API endpoints and permissions
- [ ] Check for rate limiting issues
- [ ] Review error handling configuration

Next Steps

1. **Start Simple**: Build basic workflows first
2. **Use Templates**: Learn from existing workflows
3. **Join Community**: Ask questions, share knowledge
4. **Experiment**: Try different nodes and patterns
5. **Document**: Add notes to complex workflows
6. **Monitor**: Set up error notifications
7. **Optimize**: Improve performance over time

Remember: n8n is powerful and flexible. Start with simple automations and gradually build more complex workflows as you learn!